GitHub Copilot Hackathon

■ Objective

Experience how GitHub Copilot can accelerate modern software development by building a simple, end-to-end application and using Playwright to validate it within a few hours.

Participants will use Copilot to:

- Rapidly scaffold application components (frontend + API)
- Implement core functionality using natural language prompts
- Apply basic business logic or validation to real-world scenarios
- Generate, refine, and troubleshoot meaningful Playwright end-to-end tests

Theme: Document Validation

Build an app that validates documents such as HR forms, invoices, certificates, or ID documents.

> *The goal is not to build a production-ready system, but to demonstrate how AI-assisted development can significantly reduce effort, improve productivity, and enable faster innovation.*

■ Participants

- Work individually, with optional pairing if preferred
- Optional: work in a **GitHub repository** to collect artifacts after the event

■ Pre-Requisites (Send 3+ Days Before)

Participants must have the following ready **before** the hackathon day:

- [ ] [VS Code](https://code.visualstudio.com/) installed
- [ ] [GitHub Copilot extension](https://marketplace.visualstudio.com/items?itemName=GitHub.copilot) installed and licensed
- [ ] Git installed
- [ ] Runtime of choice installed (Node.js 20+, Python 3.11+, or .NET 8+)
- [ ] Familiarity with at least one web framework

■ Agenda

Phase	Duration
----------	----------
Kickoff & Setup	30 min
Build Phase 1 — Core App	1.5 hours

Build Phase 2 — Playwright Foundations	1.5 hours
Build Phase 3 — Playwright Deepening	1.5 hours
Demos & Wrap-Up	1 hour

Kickoff & Setup (30 min)

- Welcome and session goals
- **Live demo:** Build a small feature with Copilot in 5 minutes (shows agent mode, inline completions, chat)
- **Playwright demo:** Generate a Playwright test from a user story, run it, and fix a failure with Copilot
- Overview of the challenge and example use cases
- Verify environments are working

Build Phase 1 — Core App (1.5 hours)

Goal: **Achieve a working end-to-end flow.**

- Create a simple frontend (upload form or input UI)
- Build an API endpoint to receive and process data
- Implement basic data extraction or parsing logic
- Connect frontend → API → response displayed to user
- Prepare a small `test-data/` fixture set (valid + invalid cases) for Playwright in Phase 2

  ■ ***Tip:** Use a starter template (see below) to skip boilerplate and jump straight to the interesting parts.*

Build Phase 2 — Playwright Foundations (1.5 hours)

Goal: **Create reliable Playwright foundations and hand off into deeper testing.**

- Validate fixture data from Phase 1 before writing tests
- Install and configure Playwright
- Create and stabilize one happy-path end-to-end test with Copilot
- Start one negative-path validation test so Phase 3 can go deeper immediately
- Organize test structure and note flaky selectors/timing pain points for Phase 3

Build Phase 3 — Playwright Deepening (1.5 hours)

Goal: **Deepen coverage and improve test quality.**

- Finish or expand negative-path testing from the Phase 2 handoff
- Add one richer scenario such as file upload, multi-document flow, retry/error handling, or mocked backend behavior
- Improve selectors and assertions using accessible locators such as `getByRole` and `getByLabel`
- Use trace or screenshot output to debug a failed test with Copilot
- Prepare and rehearse a clear 5-minute demo story in the final 10-15 minutes

Stretch Goals

For teams that finish early or want additional polish:

- Add unit tests using Copilot-generated test suggestions

• Support multiple document types — add a second validation scenario (e.g., invoices + HR forms) with different rules
• Accept PDF uploads — extract text from uploaded files instead of form input
• Build a summary dashboard showing all validation results
• Add data persistence (save and retrieve past validations)
• Create a reusable Copilot SKILL file — write a `.md` skill that captures your validation rules, patterns, or domain knowledge so Copilot can use it in future projects
• Add custom instructions (`.github/copilot-instructions.md`) — define project-level rules Copilot follows automatically, e.g., consistent error response format or validation patterns
• Build a custom agent (`.agent.md`) — define a specialized agent with specific instructions, e.g., a "validation expert" that knows your document schema and suggests validation patterns

Demos & Wrap-Up (1 hour)

• Team presentations (5–8 minutes each)
• Share what worked, what surprised you, and favorite Copilot moments
• Feedback survey

■ Example Use Cases

Pick **one** to keep scope manageable:

Use Case	What It Validates
----------	----------
Invoice Checker	Required fields present (vendor, amount, date, PO number), amounts are positive, date is not in the fu
HR Form Validator	Employee name, ID, department filled in; signature present; dates are logical
ID Document Verifier	Expiry date check, required fields present, format consistency
Certificate Validator	Issuer information present, valid date range, certificate number format
Expense Report Auditor	Line items sum to total, receipts attached, within policy limits

■■ Recommended Starter Stacks

Teams are free to choose their own, but here are three fast-start paths:

Stack	Frontend	Backend	Good For
-------	----------	---------	----------
JavaScript	React or plain HTML	Node.js + Express	JS/TS-comfortable teams
Python	HTML + Jinja or React	Flask / FastAPI	Data/ML-leaning teams
Java	Thymeleaf or React	Spring Boot	Java teams
.NET	Blazor or Razor Pages	ASP.NET Core Web API	.NET teams

■ Copilot Tips Cheat Sheet

Share this with participants at kickoff:

Technique	How To Use It
----------	----------
Plan Mode	Start here. Use Copilot's Plan agent to outline your approach before writing any code. e.g., *"I need
Agent Mode	Once you have a plan, use agent mode to scaffold and build features from a description. Great for i
Inline Completions	Just start typing — Copilot will suggest the next lines. Tab to accept.
Copilot Chat	Ask questions, debug errors, explain code. Use `#file` to reference specific files for context.
Generate Playwright Tests	Ask Copilot to generate a Playwright test from a user story, such as: *"Write a Playwright test for a v
Improve Selectors	Ask Copilot to replace brittle selectors with accessible ones like `getByRole` or `getByLabel`.
Debug Flaky Tests	Paste a Playwright timeout or assertion failure into chat and ask Copilot why the test is flaky and ho
Refine Assertions	Ask Copilot to strengthen a Playwright spec with clearer assertions for success, validation failures, a
Fix Errors	Paste an error message into chat and ask Copilot to diagnose and fix it.
Natural Language → Code	Write a comment describing what you want, then let Copilot generate the implementation.

■ Scope Guidance

To ensure success within the session:

- **Focus on one document type / use case** — don't try to do everything
- **Prioritize a working solution over complexity** — a simple app that works beats a complex one that doesn't
- **Treat Playwright as part of the core deliverable** — aim for at least three meaningful test scenarios
- **Use Copilot throughout** — the point is the experience, not just the output
- **Commit early and often** — it's good practice and helps you track progress
- Keep the app **simple** so there is time to go deeper on Playwright

■ What Success Looks Like

- ■ A working application (frontend + API) running locally
- ■ Meaningful Playwright coverage: happy path, negative path (at least one passing), and one deeper scenario
- ■ Clear demonstration of Copilot-assisted development during the presentation
- ■ Evidence that the team used Copilot to generate, refine, and debug Playwright tests
- ■ A rehearsed 5-minute demo flow that runs cleanly from a fresh start
- ■ A practical, business-relevant validation scenario
- ■ A creative approach and thoughtful user experience
- ■ A team that learned something new about AI-assisted development